

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử tích hợp — @SpringBootTest và Testcontainers

Buổi 7 / 12 — Môn: Kiểm thử Phần mềm

Năm học 2025–2026

Nội dung buổi học

- 1 Ôn tập buổi 6 — unit test với mock *chưa* kiểm chứng gì?
- 2 Ranh giới unit test / integration test; taxonomy test trong DigiShield
- 3 @SpringBootTest: webEnvironment, chi phí context, context caching
- 4 MockMvc: kiểm thử lớp HTTP và phân quyền
- 5 Testcontainers: vì sao H2 không đủ
- 6 Case study: TenantIsolationIT — Row-Level Security
- 7 Kiểm thử hệ event-driven (Spring Modulith) và kiểm thử kiến trúc
- 8 So sánh chiến lược kiểm thử; flaky test
- 9 Thực hành
- 10 Bài nộp & tài liệu tham khảo

Vị trí trong đề cương môn học

Chương 4 — Công cụ kiểm thử JUnit (phần mở rộng)

- Buổi 5: JUnit 5 cơ bản — lifecycle, assertions, AssertJ, parameterized
- Buổi 6: Mockito — mock / stub / verify / captor, @Nested
- **Buổi 7 (hôm nay):** từ unit test lên **kiểm thử tích hợp**:
@SpringBootTest, MockMvc, Testcontainers, test sự kiện

Kết nối với các buổi sau

- Buổi 9: API testing với Postman/Newman — kiểm thử qua HTTP *từ bên ngoài*
- Buổi 10–11: Selenium E2E — tầng cao nhất của kim tự tháp kiểm thử
- BTL: mỗi nhóm phải nộp integration test cho module của mình

Ôn buổi 6 — unit test với Mockito (đã làm)

Test thật trong modules/interception: mock repository, cô lập logic quyết định.

```
@ExtendWith(MockitoExtension.class)
class InterceptionServiceImplTest {
    @Mock AccountWatchEntryRepository watchRepo;
    @Mock InterventionEventRepository eventRepo;
    @Mock Messages messages;
    @InjectMocks InterceptionServiceImpl service;

    @Test
    void pausesWhenOnCallNewPayeeAndWatchlistHit() {
        // Stub: tai khoan dich nam trong watchlist
        when(watchRepo.findByTenantIdAndValue(any(), eq("9999")))
            .thenReturn(Optional.of(entry("9999")));
        InterventionDecision d = service.evaluate(
            req(/*onCall*/ true, /*newPayee*/ true));
        assertThat(d.decision()).isEqualTo("PAUSE");
    }
}
```

Nhanh (<10ms), ổn định, chạy không cần hạ tầng — nhưng...

Unit test với mock *chưa* kiểm chứng được gì?

Mock trả lời đúng vì...ta bảo nó trả lời như vậy

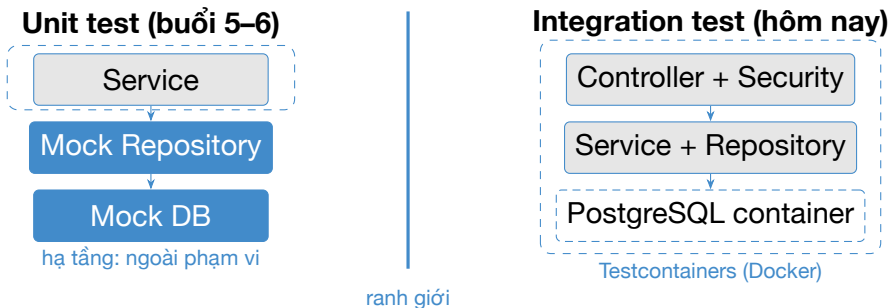
Mọi tương tác với hạ tầng đều bị thay bằng “diễn viên đóng thế”.

- **SQL / JPA mapping:** câu query sinh ra có đúng không? Cột `tenant_id` kiểu `uuid` có map được sang `UUID`?
- **Cấu hình database:** policy Row-Level Security của PostgreSQL có thực sự chặn tenant khác đọc dữ liệu? (mock không biết RLS là gì!)
- **Security:** `@PreAuthorize("hasRole('ANALYST')")` có chạy? Người chưa đăng nhập có bị chặn 401?
- **Wiring:** các bean có được Spring lắp ráp đúng? JSON serialize/deserialize qua HTTP có khớp DTO?
- **Sự kiện bất đồng bộ:** listener module khác có nhận được event?

Kết luận

Cần một tầng test **tích hợp**: chạy nhiều thành phần thật cùng nhau.

Ranh giới unit test vs integration test



Integration test kiểm chứng **sự cộng tác thật** giữa các tầng — đúng những chỗ unit test phải mock đi.

So sánh unit test và integration test

Thuộc tính	Unit test	Integration test
Phạm vi	Một class/method	Nhiều tầng cộng tác
Dependency	Mock toàn bộ	DB thật, Spring context thật
Tốc độ	< 10ms	Vài giây đến vài chục giây
Cô lập	Hoàn toàn	Phụ thuộc Docker, môi trường
Phát hiện lỗi	Logic nội tại	Mapping ORM, SQL, security, wiring
Khi fail	Nguồn gốc rõ	Có thể khó khoanh vùng
Chi phí CI	Thấp	Cao hơn (cần Docker daemon)

Khi nào *bắt buộc* cần integration test?

- Tính năng nằm ở **database** (RLS, constraint, native query)
- Cấu hình **security** (401/403), filter, serialization JSON
- Pipeline **sự kiện** xuyên module (Spring Modulith)

Taxonomy test trong DigiShield: hai tầng tách bạch (1/2)

Tầng 1 — unit test

- Thư mục: `src/test`
- Đặt tên: `*Test.java`
- Không cần Docker, chạy mọi nơi
- Lệnh: `./gradlew test`
- Ví dụ: `InterceptionServiceImplTest`, `StubAiClientTest`, `ModularityTests`

Tầng 2 — integration test

- Thư mục: `boot/app/src/integrationTest`
- Đặt tên: `*IT.java`
- Cần Docker (trừ slice test `@WebMvcTest`)
- Lệnh: `./gradlew integrationTest`
- 9 file / 30 test, ví dụ `TenantIsolationIT`, `RlsCoverageIT`, `SimulationModuleIT`, `InterceptionControllerIT...`

Taxonomy test trong DigiShield: hai tầng tách bạch (2/2)

Vì sao tách?

Vòng lặp phát triển cần phản hồi **nhANH** (unit); kiểm chứng **sâu** (IT) chạy sau, ít thường xuyên hơn — và không chặn máy thiếu Docker.

Cấu hình thật: JVM Test Suites (boot/app/build.gradle.kts)

```
testing {
    suites {
        // Suite mặc định (unit) và suite riêng cho IT
        getByName<JvmTestSuite>("test") { useJUnitJupiter() }
        register<JvmTestSuite>("integrationTest") {
            useJUnitJupiter()
            dependencies {
                implementation(project()) // dùng class sản phẩm của boot app
                implementation("org.springframework.boot:spring-boot-starter-test")
                implementation("org.springframework.modulith:spring-modulith-starter-test")
            }
            implementation(platform("org.testcontainers:testcontainers-bom:1.20.4"))
            implementation("org.springframework.boot:spring-boot-testcontainers")
            implementation("org.testcontainers:junit-jupiter")
            implementation("org.testcontainers:postgresql")
        }
    }
}
```

Cấu hình thật: thứ tự chạy và tích hợp vào check

```
targets {  
    all {  
        // IT chậm (cần Docker) chỉ chạy SAU unit test nhanh  
        testTask.configure {  
            shouldRunAfter(tasks.named("test"))  
        }  
    }  
}  
  
// `check` phải chạy CA unit test lẫn integration test  
tasks.named("check") {  
    dependsOn(testing.suites.named("integrationTest"))  
}
```

Lệnh sử dụng hằng ngày

- `./gradlew test` — vòng lặp nhanh khi code
- `./gradlew integrationTest` — trước khi mở PR (cần Docker)
- `./gradlew check` — CI: cả hai + checkstyle + JaCoCo

@SpringBootTest — khởi động ứng dụng thật để test

Ý tưởng

Annotation của Spring Boot: khởi động **toàn bộ ApplicationContext** (bean, cấu hình, filter, listener...) y như khi chạy thật, rồi cho phép @Autowired bất kỳ bean nào vào test.

- Test chạy *bên trong* JVM của ứng dụng — khác Postman (buổi 9) gọi từ bên ngoài.
- Có thể ghi đè cấu hình ngay trên annotation: `@SpringBootTest(properties = {...})`.
- Kết hợp @MockitoBean / @TestConfiguration để thay *một vài* bean khó dựng (vd JwtDecoder gọi IdP ngoài).

Trade-off

Độ tin cậy cao nhất trong JVM — nhưng khởi động context tốn **vài giây**; dùng có chọn lọc.

webEnvironment: MOCK và RANDOM_PORT

MOCK (mặc định) — không mở port thật

```
@SpringBootTest                // webEnvironment = MOCK
@AutoConfigureMockMvc
class InterventionApiIT {
    @Autowired MockMvc mockMvc;
    // Goi qua MockMvc, không có HTTP thật
}
```

RANDOM_PORT — server thật, port ngẫu nhiên

```
@SpringBootTest(webEnvironment =
    SpringBootTest.WebEnvironment.RANDOM_PORT)
class InterventionHttpIT {
    @Autowired TestRestTemplate rest;
    @LocalServerPort int port;
    // Goi HTTP thật tới localhost:port
}
```

DEFINED_PORT (dùng port khai báo trong properties, mặc định 8080): chỉ khi công cụ ngoài bắt buộc; để xung đột port trong CI.

Chi phí khởi động context và context caching

Chi phí một lần khởi động:

- Quét & khởi tạo hàng trăm bean (DigiShield: 9 module nghiệp vụ)
- Kết nối datasource, chạy migration/DDL
- Dựng security filter chain, listener
- Thực tế: **5–20 giây** mỗi context

Spring *cache* context theo cấu hình:

- Hai test class có *cùng* cấu hình (properties, profiles, MockitoBean...) ⇒ dùng lại context, chỉ trả giá 1 lần
- Khác nhau 1 property thôi ⇒ context **mới**

Hệ quả thiết kế

Chuẩn hóa cấu hình IT (dùng chung bộ properties, base class) để tối đa cache hit. Tránh @DirtiesContext nếu không thật cần — nó vứt bỏ cache.

Slice test: chỉ nạp một “lát cắt” context

@WebMvcTest — chỉ tầng web

```
@WebMvcTest(InterceptionController.class)
class InterceptionControllerIT {
    @Autowired MockMvc mockMvc;
    @MockitoBean InterceptionService service;
    // Chi load controller + JSON + MVC infrastructure
    // Service là mock -> không cần DB, rất nhanh
}
```

- Các slice khác: @DataJpaTest (chỉ JPA), @JsonTest (chỉ serialization)...
- Nhanh hơn full context nhiều vì chỉ dựng vài chục bean.

Trong DigiShield

InterceptionControllerIT (file thật) chính là một @WebMvcTest — ta mở xẻ ngay sau đây.

MockMvc: gọi “HTTP” mà không cần server

Cơ chế

MockMvc đưa request đi qua **đúng** DispatcherServlet, converter JSON, validator, (tuỳ chọn) security filter — nhưng trong bộ nhớ, không mở socket. Kết quả: kiểm chứng được toàn bộ hành vi lớp HTTP với tốc độ cao.

- `mockMvc.perform(post("/api/v1/..."))` — dựng request
- `.andExpect(status().isOk())` — assert mã trạng thái
- `.andExpect(jsonPath("$.decision").value("PAUSE"))` — assert nội dung JSON trả về

Kiểm chứng được gì mà unit test không thể?

Routing (URL đúng method?), binding `@RequestBody/ @RequestParam`, mã trạng thái, cấu trúc JSON, header, và (khi bật filter) **phân quyền**.

File thật: InterceptionControllerIT — setup (1/2)

```
boot/app/src/integrationTest/java/com/digishield/  
interception/it/
```

```
/** Web slice test cho InterceptionController. */  
@WebMvcTest(InterceptionController.class)  
@AutoConfigureMockMvc(addFilters = false)  
class InterceptionControllerIT {  
  
    @Autowired MockMvc mockMvc;  
    @Autowired ObjectMapper objectMapper;  
  
    @MockitoBean  
    InterceptionService interceptionService;  
}
```

File thật: InterceptorControllerIT — setup (2/2)

- Chỉ nạp tầng web; InterceptionService là mock $\Rightarrow$ test tập trung vào wiring request/response và (de)serialize JSON.
- `addFilters = false`: tắt security filter để chạm được controller mà không cần cấu hình OAuth2/JWT — **đổi lại** test này *không* kiểm chứng phân quyền (xem slide sau).

File thật: POST /interventions/evaluate trả về PAUSE

```
@Test
void evaluateReturnsPauseDecision() throws Exception {
    InterventionDecision pause = new InterventionDecision(
        "PAUSE",
        List.of("ON_CALL", "NEW_PAYEE", "WATCHLIST_HIT"),
        "Giao dịch đang được tạm dừng để bảo vệ bạn.");
    given(interceptionService.evaluate(any(EvaluateRequest.class)))
        .willReturn(pause);
    EvaluateRequest request = new EvaluateRequest(
        UUID.randomUUID(), new BigDecimal("50000000"),
        "9999-fraud-account", true, true);
    mockMvc.perform(post("/api/v1/interventions/evaluate")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.decision").value("PAUSE"))
        .andExpect(jsonPath("$.signals.length()").value(3))
        .andExpect(jsonPath("$.signals[0]").value("ON_CALL"));
}
```

Đọc kỹ: test này kiểm chứng điều gì?

Dòng lệnh	Điều được kiểm chứng
<code>post("/api/v1/interventions/evaluate")</code>	URL + method map đúng handler
<code>.content(objectMapper.writeValueAsString(status().isOk()))</code>	JSON body deserialize được vào record EvaluateRequest
<code>jsonPath("\$.decision")</code>	Controller trả 200, không nổ exception
<code>jsonPath("\$.signals.length()")</code>	Field của InterventionDecision serialize đúng tên
<code>jsonPath("\$.signals[0]")</code>	Mảng signals giữ nguyên 3 phần tử
	Phần tử đầu là ON_CALL — thứ tự được giữ

Lưu ý: logic PAUSE/WARN/ALLOW *không* được kiểm chứng ở đây (service là mock) — việc đó thuộc về unit test buổi 6. Mỗi tầng test một trách nhiệm.

Kiểm thử phân quyền: đặc tả của DigiShield

Quy tắc trên InterceptorController (code thật)

```
@PreAuthorize("isAuthenticated()")  
@PostMapping("/api/v1/interventions/evaluate") ...
```

```
@PreAuthorize("hasRole('ANALYST')")  
@GetMapping("/api/v1/interventions") ...
```

Ai gọi GET /interventions	Kỳ vọng	Loại test
Chưa đăng nhập	401 Unauthorized	tiêu cực
Đăng nhập, role LEARNER	403 Forbidden	tiêu cực
Đăng nhập, role ANALYST	200 OK	tích cực

Test phân quyền **tiêu cực** là bắt buộc: lỗi “quên @PreAuthorize” không bao giờ lộ ra ở happy path.

Kiểm thử phân quyền với MockMvc (bật security filter)

```
@SpringBootTest          // không dùng profile dev!
@AutoConfigureMockMvc    // giữ nguyên security filter chain
class InterventionAuthzIT {
    @Autowired MockMvc mockMvc;
    @MockitoBean JwtDecoder jwtDecoder; // chặn gọi IdP thật
    @Test
    void anonymousGets401() throws Exception {
        mockMvc.perform(get("/api/v1/interventions"))
            .andExpect(status().isUnauthorized());
    }
    @Test
    void learnerGets403() throws Exception {
        mockMvc.perform(get("/api/v1/interventions")
            .with(jwt().authorities(
                new SimpleGrantedAuthority("ROLE_LEARNER"))))
            .andExpect(status().isForbidden());
    }
}
```

jwt () (spring-security-test) giả lập một JWT đã xác thực với authority tùy chọn
— không cần IdP thật.

Cảnh báo: đừng test phân quyền trên profile dev

Cấu hình thật của DigiShield

- Profile dev dùng `DevSecurityConfig: permitAll()` mọi request, còn `@PreAuthorize` bị **vô hiệu** (method security chỉ bật ở `@Profile("!dev")`).
- `DevTenantFilter` lấy tenant từ header `X-Tenant-Id`, thiếu thì fallback tenant demo — dev **không** bao giờ tạo ra 401/403.
- Test phân quyền trên dev sẽ **pass ảo**: ai gọi gì cũng 200.
- Muốn 401/403 thật: chạy chuỗi security *production* (`@Profile("!dev")`) và stub `JwtDecoder` như slide trước — đúng cách `TenantIsolationIT` đã làm.

Bài học tổng quát

Integration test chỉ có giá trị khi chạy trên **cấu hình giống production**.
Test trên cấu hình “dễ dãi” là kiểm chứng một hệ thống không tồn tại.

Vì sao H2 in-memory không đủ?

H2 tiện — nhưng không phải PostgreSQL

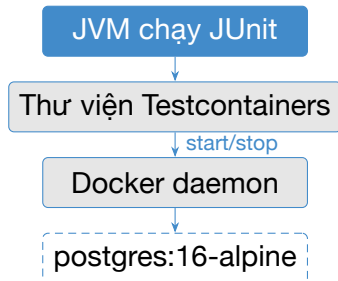
Profile dev của DigiShield chạy H2 để khởi động nhanh, không cần Docker. Với *integration test* thì H2 tạo ra khoảng mù nguy hiểm.

- **Row-Level Security là tính năng của PostgreSQL** — H2 không có CREATE POLICY, không có `current_setting()`. Cơ chế cách ly tenant — xương sống bảo mật của DigiShield — **không thể** kiểm chứng trên H2.
- Khác dialect: kiểu dữ liệu (`uuid`, `timestampz`, `jsonb`), hàm, ngữ nghĩa transaction, native query.
- Flyway migration viết cho PostgreSQL có thể chạy khác (hoặc fail) trên H2.

Nguyên tắc

Test với hạ tầng **cùng loại, cùng phiên bản** với production. Câu trả lời hiện đại: **Testcontainers**.

Testcontainers: hạ tầng thật, vòng đời do test quản lý



Vòng đời:

- 1 Test class khởi động
- 2 Pull image (lần đầu) rồi start container, gán **port ngẫu nhiên**
- 3 Test chạy trên PostgreSQL thật
- 4 JVM kết thúc: container bị dừng và dọn sạch (container phụ *Ryuk* bảo đảm cả khi test crash)

Yêu cầu

Docker daemon phải đang chạy; nếu không: `ContainerLaunchException`.

@Testcontainers và @Container

```
@Testcontainers // JUnit 5 extension
class RlsCoverageIT { // file that của DigiShield

    @Container // TC quan ly vong doi field nay
    static final PostgreSQLContainer<?> POSTGRES =
        new PostgreSQLContainer<>("postgres:16-alpine");
    ...
}
```

- Field static: **một container cho cả class** (khởi động 1 lần, các @Test dùng chung — nhanh, nhưng nhớ dọn dữ liệu).
- Field không static: container mới cho *mỗi* test — sạch nhưng chậm.
- TenantIsolationIT chọn cách thứ ba: tự `POSTGRES.start()` trong khối static `{}` để container sẵn sàng *trước khi* Spring context được dựng.

@DynamicPropertySource: nối Spring vào container

Vấn đề

Port của container là **ngẫu nhiên** mỗi lần chạy — không thể hardcode `spring.datasource.url` trong `application.yml`.

```
@DynamicPropertySource // chạy TRƯỚC khi context được tạo
static void datasourceProperties(
    DynamicPropertyRegistry registry) {
    registry.add("spring.datasource.url",
        POSTGRES::getJdbcUrl); // lấy port that
    registry.add("spring.datasource.username",
        POSTGRES::getUsername);
    registry.add("spring.datasource.password",
        POSTGRES::getPassword);
}
```

Cách gọn hơn: @ServiceConnection

Với `spring-boot-testcontainers`, đặt `@ServiceConnection` lên container là Spring tự suy ra url/username/password — khi không cần tùy biến gì thêm.

Bối cảnh: DigiShield là hệ multi-tenant

- Nhiều tổ chức (tenant) — cơ quan A, ngân hàng B... — dùng **chung một database**, chung bảng `phishing_report`.
- Yêu cầu an ninh tuyệt đối: tenant A **không bao giờ** được thấy báo cáo lừa đảo của tenant B.
- Lọc `WHERE tenant_id = ?` trong code là chưa đủ: một repository method “quên” filter, một native query — và dữ liệu lộ chéo.

phishing_report

id=... tenant_id=A
id=... tenant_id=B
id=... tenant_id=A

RLS policy: phiên của tenant A chỉ “nhìn thấy” các dòng của A

Giải pháp của DigiShield

Đẩy việc cách ly xuống **database**: PostgreSQL **Row-Level Security (RLS)** — database tự lọc dòng theo tenant của phiên, bất kể ứng dụng viết query thế nào.

RLS hoạt động thế nào? (file thật `it/rls-setup.sql`)

```
ALTER TABLE phishing_report ENABLE ROW LEVEL SECURITY;  
-- Áp dụng cả với table owner (superuser/BYPASSRLS van bỏ qua):  
ALTER TABLE phishing_report FORCE ROW LEVEL SECURITY;  
  
CREATE POLICY tenant_isolation ON phishing_report  
  USING (tenant_id = NULLIF(  
    current_setting('app.tenant_id', true), '')::uuid)  
  WITH CHECK (tenant_id = NULLIF(  
    current_setting('app.tenant_id', true), '')::uuid);
```

- `app.tenant_id` là một **GUC** (biến cấu hình của phiên PostgreSQL).
Production: aspect `RlsTenantAspect` gọi `set_config('app.tenant_id', ?, true)` đầu mỗi transaction.
- **USING**: lọc dòng khi **SELECT/UPDATE/DELETE**; **WITH CHECK**: chặn **INSERT** “gắn nhãn” tenant khác.
- Mọi query — kể cả query quên filter — đều bị policy áp lên **trước** mệnh đề **WHERE** của ứng dụng.

Hai chi tiết quyết định chất lượng của test

1. Vì sao phải FORCE ROW LEVEL SECURITY?

Mặc định PostgreSQL *bỏ qua* RLS cho **table owner**; FORCE buộc policy áp cả lên owner — nhưng *không* chạm được superuser hay role có BYPASSRLS. User mặc định của Testcontainers lại là **chủ bảng kiêm superuser**, nên một mình FORCE chưa đủ: test còn phải kết nối bằng `normal_user NOSUPERUSER NOBYPASSRLS` (slide sau). Thiếu hai lớp này, mọi dòng đều hiện ra, test cách ly **pass một cách vô nghĩa** — lỗi kinh điển của test RLS.

2. Fail-closed: thiếu tenant thì thấy 0 dòng

Khi `app.tenant_id` chưa được set: `current_setting(..., true)` trả về NULL $\Rightarrow$ vị từ `tenant_id = NULL` không bao giờ đúng $\Rightarrow$ **không dòng nào** hiển thị. Quên set tenant $\neq$ lộ toàn bộ dữ liệu — hệ thống “đóng an toàn”.

Một test tốt phải kiểm chứng **cả hai** hành vi này — và `TenantIsolationIT` làm đúng như vậy.

TenantIsolationIT (1/6): cấu hình context

```
@SpringBootTest(properties = {
    // Test tu quan ly schema bang script SQL riêng
    "spring.jpa.hibernate.ddl-auto=none",
    "spring.flyway.enabled=false",
    "spring.jpa.properties.hibernate.dialect=" +
        "org.hibernate.dialect.PostgreSQLDialect",
    "spring.datasource.driver-class-name=org.postgresql.Driver",
    // IT nay khong can Redis: cache no-op, tat health check
    "spring.cache.type=none",
    "management.health.redis.enabled=false" })
class TenantIsolationIT {

    @TestConfiguration
    @EnableAspectJAutoProxy
    static class TestConfig {
        @Bean JwtDecoder jwtDecoder() {           // stub, khong gọi IdP
            return Mockito.mock(JwtDecoder.class);
        }
    }
    ...
}
```

TenantIsolationIT (2/6): container và user thường

```
static final PostgreSQLContainer<?> POSTGRES =
    new PostgreSQLContainer<>("postgres:16-alpine");
static { POSTGRES.start(); } // trước khi context được dùng

@DynamicPropertySource
static void datasourceProperties(DynamicPropertyRegistry reg) {
    try (var conn = DriverManager.getConnection(
        POSTGRES.getJdbcUrl(), POSTGRES.getUsername(),
        POSTGRES.getPassword());
        var stmt = conn.createStatement()) {
        // Tao role KHÔNG superuser, KHÔNG bypass RLS
        stmt.execute("CREATE ROLE normal_user WITH LOGIN "
            + "PASSWORD 'normal_pass' NOSUPERUSER NOBYPASSRLS");
        stmt.execute("GRANT ALL ON SCHEMA public TO normal_user");
    }
    reg.add("spring.datasource.url", POSTGRES::getJdbcUrl);
    reg.add("spring.datasource.username", () -> "normal_user");
    reg.add("spring.datasource.password", () -> "normal_pass");
}
```

Ứng dụng kết nối bằng `normal_user` — đúng như production không bao giờ chạy bằng `superuser`.

TenantIsolationIT (3/6): schema sạch trước mỗi test

```
@Autowired PhishingReportRepository reportRepository;
@Autowired TransactionTemplate transactionTemplate;
@Autowired DataSource dataSource;
@BeforeEach
void setUpSchema() throws Exception {
    // Tao lai bang + policy: du lieu sach, RLS chac chan bat
    try (var connection = dataSource.getConnection()) {
        ScriptUtils.executeSqlScript(connection,
            new ClassPathResource("it/rls-setup.sql"));
    }
}
@AfterEach
void clearTenant() {
    TenantContext.clear();    // khong ro ri tenant sang test khac
}
```

- Tái tạo bảng mỗi test ⇒ **test độc lập**, không phụ thuộc thứ tự — vũ khí chống flaky số một.
- DDL của script được giữ *đồng bộ* với migration Flyway thật.

TenantIsolationIT (4/6): helper đóng vai tenant

```
// Chạy `work` trong 1 transaction với tenant cho trước:
private <T> T runAsTenant(UUID tenantId, Supplier<T> work) {
    TenantContext.set(tenantId.toString());
    try {
        return transactionTemplate.execute(status -> work.get());
    } finally {
        TenantContext.clear();
    }
}

private UUID saveReportForTenant(UUID tenantId) {
    return runAsTenant(tenantId, () -> {
        PhishingReport report = new PhishingReport(
            UUID.randomUUID(), tenantId, UUID.randomUUID(),
            "suspicious-email-payload", AiLabel.THREAT,
            0.97, ReportStatus.SUBMITTED);
        return reportRepository.save(report).getId();
    });
}
```

TenantIsolationIT (5/6): tenant A không đọc được của B

```
private static final UUID TENANT_A =
    UUID.fromString("11111111-1111-1111-1111-111111111111");
private static final UUID TENANT_B =
    UUID.fromString("22222222-2222-2222-2222-222222222222");
@Test
void rlsIsolatesReportsBetweenTenants() {
    UUID reportA = saveReportForTenant(TENANT_A);
    UUID reportB = saveReportForTenant(TENANT_B);
    // Tenant A chỉ thay của mình; B cũng vậy
    List<PhishingReport> seenByA =
        runAsTenant(TENANT_A, () -> reportRepository.findAll());
    assertThat(seenByA).extracting(PhishingReport::getId)
        .containsExactly(reportA);
    List<PhishingReport> seenByB =
        runAsTenant(TENANT_B, () -> reportRepository.findAll());
    assertThat(seenByB).extracting(PhishingReport::getId)
        .containsExactly(reportB)
        .doesNotContain(reportA);
}
```

Điểm đắt giá nhất: cố tình truy vấn chéo tenant

```
// Tenant B CÓ TINH query dịch danh dữ liệu của tenant A:  
List<PhishingReport> crossTenant = runAsTenant(TENANT_B,  
    () -> reportRepository.findById(TENANT_A));  
  
assertThat(crossTenant).isEmpty();
```

Vì sao đây là assertion giá trị nhất?

Query có hắc WHERE `tenant_id = :tenantA` — nếu cách ly chỉ nằm ở tầng ứng dụng thì nó *phải* trả về dữ liệu của A. Kết quả rỗng chứng minh RLS lọc **trước** cả mệnh đề WHERE: dù code bị lừa (hoặc có lỗi), database vẫn không nhả dữ liệu chéo tenant.

Tư duy thiết kế test

Đừng chỉ test “đường ngay”: hãy đóng vai **kẻ tấn công** / đoạn code lỗi và chứng minh hàng rào vẫn giữ. Đây là test bảo mật ở mức tích hợp.

TenantIsolationIT (6/6): fail-closed khi thiếu tenant

```
@Test
void queryingWithoutTenantContextFailsClosed() {
    saveReportForTenant(TENANT_A);    // co du lieu san
    // (1) Duong qua aspect: thieu tenant -> fail NGAY
    TenantContext.clear();
    assertThatThrownBy(() -> transactionTemplate.execute(
        status -> reportRepository.findAll()))
        .isInstanceOf(IllegalStateException.class);
    // (2) Duong SQL thuan (khong qua aspect):
    //     GUC chua set -> policy che het -> 0 dong
    long visible = countRowsWithoutTenantGuc();
    assertThat(visible).isZero();
}

private long countRowsWithoutTenantGuc() { // JDBC tho
    try (var c = dataSource.getConnection();
        var st = c.createStatement();
        var rs = st.executeQuery(
            "SELECT count(*) FROM phishing_report")) {
        rs.next(); return rs.getLong(1);
    } catch (Exception e) { throw new IllegalStateException(e); }
}
```

Bài học từ TenantIsolationIT

- 1 **Test điều mock không thể chứng minh:** RLS sống trong PostgreSQL — chỉ PostgreSQL thật (Testcontainers) mới kiểm chứng được.
- 2 **Vô hiệu hóa các lỗi tắt làm test pass ảo:** FORCE ROW LEVEL SECURITY + kết nối bằng `normal_user NOBYPASSRLS` — nếu không, mọi assertion đều vô nghĩa.
- 3 **Hai phía của thuộc tính an ninh:** cách ly khi có tenant, và fail-closed khi *thiếu* tenant.
- 4 **Kiểm chứng ở nhiều tầng:** qua JPA repository (đường thật của ứng dụng) và qua JDBC thô (chứng minh hàng rào nằm ở DB, không phải ở aspect).
- 5 **Test độc lập:** tái tạo schema mỗi test, luôn `TenantContext.clear()` trong `finally/@AfterEach`.

Test “vệ sĩ” đi kèm: RlsCoverageIT

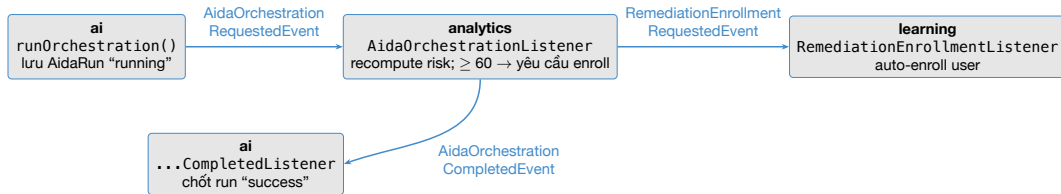
Chạy migration Flyway thật lên PostgreSQL dùng một lần, rồi soi catalog:

```
@Test
void everyTenantScopedTableHasRlsEnabledAndAPolicy() {
    Flyway.configure().dataSource(POSTGRES.getJdbcUrl(), ...)
        .locations("classpath:db/migration").load().migrate();
    Set<String> tenantTables = query(c, // bang co cot tenant_id
        "SELECT table_name FROM information_schema.columns "
        + "WHERE column_name = 'tenant_id'");
    Set<String> rlsEnabled = query(c, "... c.relrowsecurity");
    Set<String> withPolicy = query(c,
        "SELECT tablename FROM pg_policies ...");
    // Moi bang tenant-scoped PHAI bat RLS va co policy
    assertThat(missing).isEmpty();
}
```

Ý nghĩa

Ai đó thêm bảng mới có tenant_id mà quên RLS $\Rightarrow$ test fail và *nêu đích danh* bảng thiếu. Một quy tắc kiến trúc dữ liệu được thực thi tự động, mãi mãi.

Pipeline sự kiện AIDA: bốn module bắt tay qua event



- `@ApplicationModuleListener` (Spring Modulith): chạy **bất đồng bộ**, transaction riêng, tenant lấy từ event.
- Không module nào gọi thẳng module nào — chỉ giao tiếp qua event trong contracts.

Test hệ event-driven: hai mức, hai câu hỏi

Mức 1: listener đơn lẻ = unit test

- Gọi thẳng `listener.on(event)` với mock dependency
- DigiShield có sẵn: `AidaOrchestrationListenerTest`, `AidaOrchestrationCompletedListenerTest`
- Trả lời: “*nhận được event thì xử lý đúng không?*”

Mức 2: kịch bản xuyên module = IT

- Context thật + DB thật; bắt event bằng listener cài trong test
- Trả lời: “*event có thực sự được phát và định tuyến tới module kia không?*”
- Ví dụ thật: `SimulationModuleIT`

Lỗi chỉ mức 2 bắt được: quên `publish`, sai kiểu event, listener không được đăng ký, nhầm điều kiện phát (CLICK vs DELIVERED) — unit test từng listener vẫn xanh, pipeline vẫn đứt.

File thật: SimulationModuleIT — bắt event trong test

```
@SpringBootTest(properties = {
    "spring.jpa.hibernate.ddl-auto=create-drop",
    "spring.flyway.enabled=false", ... })
class SimulationModuleIT {
    static final PostgreSQLContainer<?> POSTGRES =
        new PostgreSQLContainer<>("postgres:16-alpine");
    static { POSTGRES.start(); }
    // Listener cài riêng cho test: gom mọi event phát ra
    static class TestEventListener {
        private final List<Object> events =
            new CopyOnWriteArrayList<>();
        @EventListener
        public void onEvent(Object event) { events.add(event); }
        public <T> List<T> ofType(Class<T> type) {
            return events.stream().filter(type::isInstance)
                .map(type::cast).toList();
        }
    }
}
```

Kỹ thuật “event probe”: một bean `@TestConfiguration` lắng nghe tất cả event để test assert lên chúng.

File thật: CLICK phát event — DELIVERED thì không

```
@Test
void recordingClickPublishesUserClickedEvent() {
    TenantContext.set(TENANT);
    UUID userId = UUID.randomUUID();
    SimCampaign campaign = simulationService
        .createCampaign(Channel.EMAIL, null, null);
    simulationService.recordEvent(
        campaign.getId(), userId, SimAction.CLICK);
    var published = testEventListener.ofType(UserClickedSimulationEvent.class);
    assertThat(published).hasSize(1);
    assertThat(published.get(0).userId()).isEqualTo(userId);
}

@Test
void recordingNonClickPublishesNoEvent() {
    TenantContext.set(TENANT);
    SimCampaign c = simulationService
        .createCampaign(Channel.SMS, null, null);
    simulationService.recordEvent(
        c.getId(), UUID.randomUUID(), SimAction.DELIVERED);
    assertThat(testEventListener
        .ofType(UserClickedSimulationEvent.class)).isEmpty();
}
```

Kiểm thử *kiến trúc*: ModularityTests (file thật)

boot/app/src/test/java/com/digishield/ModularityTests.java —
chạy trong tầng *unit* (không cần Docker):

```
class ModularityTests {  
    private final ApplicationModules modules =  
        ApplicationModules.of(DigishieldApplication.class);  
  
    @Test  
    void verifiesModularStructure() {  
        modules.verify();    // pham ranh gioi module -> FAIL  
    }  
  
    @Test  
    void writesDocumentation() {    // sinh so do C4/PlantUML  
        new Documenter(modules).writeDocumentation()  
            .writeIndividualModulesAsPlantUml();  
    }  
}
```

Ý nghĩa: nếu ai đó để *simulation* import thẳng class nội bộ của *auth*,
verify() fail ngay — quy tắc kiến trúc trở thành một **test case**, không phải lời
dẫn trong tài liệu.

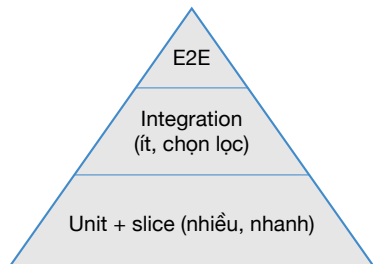
So sánh bốn chiến lược kiểm thử backend

Chiến lược	Tốc độ	Độ tin cậy	Phạm vi	Dùng khi
Unit + Mockito	<10ms	cao nhất về logic, mù về hạ tầng	1 class	logic nghiệp vụ, nhánh rẽ, biên
Slice @WebMvcTest + MockMvc	~1–2s	wiring HTTP thật, service mock	tầng web	routing, JSON, mã trạng thái
@SpringBootTest + H2	~5–10s	wiring đủ, DB “giả giọng”	cả app, DB khác loại	luồng CRUD chung, không đụng tính năng PG
@SpringBootTest + Testcontainers PG	~10–30s	cao nhất trong JVM	cả app + DB thật	RLS, migration, native query, event xuyên module

Nguyên tắc chọn

Dùng chiến lược **nhẹ nhất** đủ kiểm chứng điều cần kiểm chứng — leo lên tầng nặng hơn chỉ khi tầng dưới “mù” về rủi ro đó.

Kim tự tháp: đa số unit, ít integration test chất lượng cao



DigiShield áp dụng đúng tinh thần này:

- 284 unit test trong `src/test` các module — phủ logic nhánh, biên, exception
- Chỉ **9 file IT** (30 test) trong `src/integrationTest` — mỗi file gác một rủi ro *không thể* kiểm chứng cách khác: RLS, độ phủ RLS, event xuyên module, wiring HTTP, security header, CSP

Câu hỏi trước khi viết một IT mới

“Rủi ro nào unit test *không thể* bắt được mà test này bắt được?” — nếu không trả lời được, viết unit test đi.

Flaky test: kẻ thù số một của integration test

Flaky = lúc pass lúc fail mà code không đổi

Một suite flaky bị cả team *mất niềm tin* — “chắc lại flaky thôi” — và lỗi thật sẽ lọt qua.

- **Thời gian / bất đồng bộ:** assert ngay sau khi phát event, nhưng `@ApplicationModuleListener` chạy trên thread khác — lúc kịp, lúc không.
- **Thứ tự test:** test A để lại dữ liệu, test B vô tình phụ thuộc; đổi thứ tự chạy là fail.
- **Trạng thái chia sẻ:** container/DB dùng chung giữa các test mà không dọn; `TenantContext` (`ThreadLocal`) rò rỉ sang test sau.
- **Môi trường:** port bị chiếm, đồng hồ hệ thống, mạng chậm khi pull image.

Phòng chống flaky — các kỹ thuật DigiShield đang dùng

Kỹ thuật	Trong code thật
Dọn / tái tạo trạng thái mỗi test	TenantIsolationIT chạy lại rls-setup.sql trong @BeforeEach
Dọn ThreadLocal trong @AfterEach	TenantContext.clear() ở cả hai IT
Không ngủ chờ (Thread.sleep)	SimulationModuleIT định tuyến publisher về gọi đồng bộ trong test; nơi khác dùng cơ chế chờ theo điều kiện (Awaitility)
Port ngẫu nhiên, không hardcode	Testcontainers + @DynamicPropertySource
Collection an toàn đa luồng cho probe	CopyOnWriteArrayList trong TestEventListener

Quy tắc vàng: test phải **tự lo** tiền điều kiện của nó và **trả lại** môi trường như cũ — không tin tưởng test chạy trước, không để lại “di sản” cho test chạy sau.

Thực hành trên lớp

BT	Nội dung	Kỹ thuật	Thời gian
1	Chạy ./gradlew integrationTest, đọc report HTML	Gradle, Testcontainers	15 phút
2	Đọc hiểu TenantIsolationIT: trả lời 3 câu hỏi kiểm tra	RLS, fail-closed	15 phút
3	Viết IT mới: luồng watchlist POST → GET check	@SpringBootTest + MockMvc	30 phút

Chuẩn bị: Docker Desktop đang chạy; pull image trước cho nhanh: `docker pull postgres:16-alpine`

3 câu hỏi của BT2

(1) Bỏ FORCE ROW LEVEL SECURITY thì test nào pass ảo, vì sao? (2) Vì sao phải tạo normal_user? (3) Chuyện gì xảy ra nếu quên `TenantContext.clear()`?

BT1 — chạy và đọc kết quả integration test

```
cd digishield
# 1) Unit test (nhANH, không cần Docker)
./gradlew test

# 2) Integration test (cần Docker; lần đầu pull image)
./gradlew integrationTest

# 3) Mở report HTML
open boot/app/build/reports/tests/integrationTest/index.html
```

- Ghi lại: tổng thời gian của test vs integrationTest; thời gian từng test class.
- Quan sát log: container PostgreSQL khởi động, port được gán.
- Thử tắt Docker rồi chạy lại — đọc hiểu ContainerLaunchException.
- So sánh: chạy integrationTest lần 2 nhanh hơn bao nhiêu (image đã cache)?

BT3 — IT mới: luồng watchlist (khung sườn)

ANALYST thêm tài khoản đáng ngờ rồi tra cứu phải thấy `inWatchlist=true` — file

`.../WatchlistFlowIT.java:`

```
@SpringBootTest(properties = {
    "spring.jpa.hibernate.ddl-auto=create-drop",
    "spring.flyway.enabled=false",
    "spring.cache.type=none", "management.health.redis.enabled=false" })
@AutoConfigureMockMvc(addFilters = false) // don gian hoa: bo qua authz
class WatchlistFlowIT {
    static final PostgreSQLContainer<?> POSTGRES =
        new PostgreSQLContainer<>("postgres:16-alpine");
    static { POSTGRES.start(); }
    @Autowired MockMvc mockMvc;
    @DynamicPropertySource
    static void ds(DynamicPropertyRegistry reg) {
        reg.add("spring.datasource.url", POSTGRES::getJdbcUrl);
        reg.add("spring.datasource.username", POSTGRES::getUsername);
        reg.add("spring.datasource.password", POSTGRES::getPassword);
    }
    // + @TestConfiguration stub JwtDecoder (nhu TenantIsolationIT)
    // + TenantContext: set tenant demo trong @BeforeEach
}
```

BT3 — thân test cần hoàn thiện

```
@Test
void addedAccountIsFlaggedByCheck() throws Exception {
    // 1) POST thêm tài khoản vào watchlist -> 201 Created
    mockMvc.perform(post("/api/v1/account-watchlist")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"type": "bank_account",
             "value": "9704-2222-9999",
             "risk_level": "high",
             "source": "A05"}"""))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.value")
            .value("9704-2222-9999"));
    // 2) GET check -> phải thay inWatchlist = true
    mockMvc.perform(get("/api/v1/account-watchlist/check")
        .param("value", "9704-2222-9999"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.inWatchlist").value(true))
        .andExpect(jsonPath("$.riskLevel").value("HIGH"));
}
```

Nâng cao (điểm cộng): thêm test tiêu cực — check một giá trị *chưa* thêm phải trả `inWatchlist=false`.

Lỗi thường gặp khi làm bài

Triệu chứng	Nguyên nhân / cách xử lý
ContainerLaunchException	Docker chưa chạy → mở Docker Desktop
Treo lâu ở lần chạy đầu No qualifying bean JwtDecoder	Đang pull image → docker pull trước Full context cần security → stub JwtDecoder bằng @TestConfiguration
IllegalStateException thiếu tenant	Chưa TenantContext.set(...) trước khi gọi service ghi DB
Đặt file *IT.java vào src/test	Sai source set → không được task integrationTest nhặt; chuyển sang src/integrationTest
Test pass một mình, fail khi chạy cả suite	Rò rỉ trạng thái → dọn dữ liệu / ThreadLocal trong @BeforeEach/@AfterEach

Bài nộp (về nhà, tính vào BTL — A1.2, 30%)

Yêu cầu cho nhóm BTL

- 1 Hoàn thiện WatchlistFlowIT (BT3) chạy pass, kèm ảnh report HTML.
- 2 Viết **ít nhất 2 integration test** cho *module nhóm mình phụ trách* trong BTL:
 - 1 test luồng HTTP qua MockMvc (tích cực + tiêu cực), và
 - 1 test chạm hạ tầng thật (Testcontainers PostgreSQL) hoặc test sự kiện xuyên module (kiểu SimulationModuleIT).
- 3 Nửa trang phân tích: mỗi IT gác rủi ro gì mà unit test của nhóm (buổi 5–6) không bắt được? Đo và so sánh thời gian chạy hai tầng test.

Nộp

Push lên branch `btl/nhom-XX` + báo cáo PDF trước buổi 8. Test phải chạy được bằng `./gradlew integrationTest`.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., John Wiley & Sons, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] Testcontainers (guides, module PostgreSQL). <https://testcontainers.com/>
- [6] Spring Boot Testing.
<https://docs.spring.io/spring-boot/reference/testing/>
- [7] Spring Modulith (testing, `ApplicationModules.verify()`).
<https://docs.spring.io/spring-modulith/reference/>
- [8] PostgreSQL Row Security Policies.
<https://www.postgresql.org/docs/16/ddl-rowsecurity.html>

Tổng kết buổi 7

Nhớ 5 điều:

- 1 Unit test mù về hạ tầng — IT lắp đúng khoảng mù đó.
- 2 DigiShield tách 2 tầng: `src/test` (*Test) vs `src/integrationTest` (*IT, cần Docker).
- 3 `@SpringBootTest` dắt — tận dụng context caching, slice test khi đủ.
- 4 Tính năng của database (RLS) phải test trên database thật: Testcontainers.
- 5 Ít IT thôi, nhưng mỗi cái phải gác một rủi ro không thể kiểm chứng cách khác.

Buổi 8

Chuyển sang **kiểm thử hộp đen** (Chương 5): phân vùng tương đương, phân tích giá trị biên, bảng quyết định, chuyển trạng thái — thiết kế test case từ đặc tả, áp lên chính các API vừa test hôm nay.

Hỏi — đáp